

PA3b: Sentiment Classification

Jared Ng
PA3b 4
jaredn@chalmers.se

Zheng Wei Liao
PA3b 4
liao@chalmers.se

1 Introduction

This report will be about a sentiment classification experiment that is done in our company, "Birch", and our goal is to develop a system capable of text classification. In this experiment, we will look at various models and tune their hyper parameters to obtain the best model that can perform sentiment analysis classification. We will be comparing each model using their accuracy achieved in the test set. We will also look at some of the limitations that we encountered while doing this project.

2 Methodology

In this section, we will look at how we represent our data, choose our algorithm, and determine which is the better model.

2.1 Data

In order for the model to read and understand our data, we converted the CSV file containing our data into DataFrames and utilized Term Frequency-Inverse Document Frequency (TF-IDF) to obtain the weights for each feature and Bidirectional Encoder Representations from Transformers (BERT) word embeddings are used to obtain weights for features using contextual clues. We processed our data by removing trailing and leading white spaces, and change wrongly labeled sentiments to either 'positive', 'negative' or 'neutral'. We split the crowd sourced data and gold data into 80% training data and 20% validation data. We would use this validation set to compare the models before evaluating the best model obtained on the given test set with trusted labels.

2.2 Model

For our experiment, we have chosen to use Multinomial Naive Bayes, Random Forest and Logistic Regression models. Furthermore, we utilized the GridSearchCV function from scikit-learn to help

obtain the best hyper parameters that we can use for our models in this experiment. We will first use GridSearchCV on the training data and obtain the best hyper parameters for each models by comparing each models on the validation set we have split from the given train set. Next, we test the model with the highest accuracy on the test set that was given with the trusted labels.

2.2.1 Logistic Regression Model

Table 11 shows the hyper parameters that will be tuned using GridSearchCV

Hyper Parameters	Choices
C	0.01, 0.1, 1, 10, 100
penalty	'l1', 'l2'
class_weight	None, 'balanced'
solver	'liblinear'

Table 1: Various Hyper Parameters

2.2.2 Random Forest Classifier

Table 11 shows the hyper parameters that will be tuned using GridSearchCV

Hyper Parameters	Choices
n_estimators	10, 50, 100
max_depth	None, 5, 10, 15
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 4
bootstrap	True, False

Table 2: Various Hyper Parameters

2.2.3 Multinomial Naive Bayes

Table 11 shows the hyper parameters that will be tuned using GridSearchCV

2.3 Method Choices and Reasoning

We have chosen to use TF-IDF with unigrams instead of bigrams, trigrams, and other N-grams due

Hyper Parameters	Choices
alpha	0.01, 0.1, 1, 10
fit_prior	True, False

Table 3: Various Hyper Parameters

to our limited time and resources, a large feature file would be generated with questionable improvements and if there are improvements, it could most probably be marginal. For our Multinomial Naive Bayes model, we would only use TF-IDF because of the negative weights of the BERT word embeddings. Our reasoning for choosing these 3 models for our experiment is because of the constraint of limited time and resources, as more complex models would require more time for training the models or it would require better resources that can reduce our training time. Due to these limitations, we also used GridSearchCV to help us obtain the best combination of hyper parameters for each individual model.

3 Results

We will be comparing the various models' accuracy by using a confusion matrix.

3.1 Logistic Regression Model

We found that the best hyper parameters for the Logistic Regression Model are 'C': 0.1, 'class_weight': None, 'penalty': 'l2', 'solver': 'liblinear' for both training sets.

3.1.1 Crowd sourced Data

Figure 1 shows the confusion matrix of the Logistic Regression Model on the validation set after training the model on the crowd sourced data.

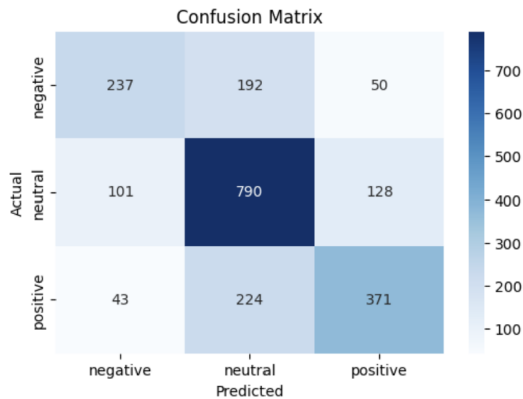


Figure 1: Logistic Regression Crowd Sourced Data Confusion Matrix

Table 11 shows the classification report of the Logistic Regression Model of the validation set after training on the crowd sourced data.

Labels	Precision	Recall	F1 Score
negative	0.62	0.49	0.55
neutral	0.66	0.78	0.71
positive	0.68	0.58	0.63

Table 4: Classification Report

We obtained an accuracy score of 65.45% after training the Logistic Regression model on the crowd sourced data.

3.1.2 Gold Data

Figure 2 shows the confusion matrix of the Logistic Regression model on the validation set after training the model on the gold data.

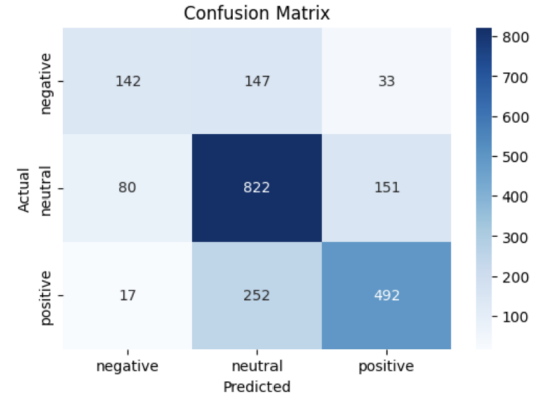


Figure 2: Logistic Regression Gold Data Confusion Matrix

Table 11 shows the classification report of the Logistic Regression Model of the validation set after training on the gold data.

Labels	Precision	Recall	F1 Score
negative	0.59	0.44	0.51
neutral	0.67	0.78	0.72
positive	0.73	0.65	0.68

Table 5: Classification Report

We obtained an accuracy score of 68.16% after training the Logistic Regression model on the gold data.

3.2 Random Forest Classifier

We found that the best hyper parameters for the Random Forest Classifier are 'bootstrap':

False, 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100 for the crowd sourced training set and 'bootstrap': False, 'max_depth': 15, 'min_samples_leaf': 2, 'min_samples_split': 10, 'n_estimators': 100 for the gold training set.

3.2.1 Crowd sourced Data

Figure 3 shows the confusion matrix of the Random Forest model on the validation set after training the model on the crowd sourced data.

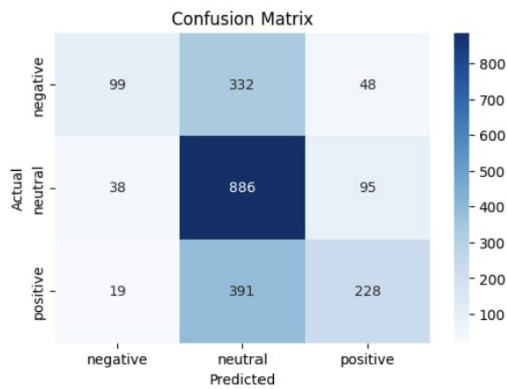


Figure 3: Random Forest Crowd Sourced Data Confusion Matrix

Table 11 shows the classification report of the Random Forest Classifier Model of the validation set after training on the crowd sourced data.

Labels	Precision	Recall	F1 Score
negative	0.63	0.21	0.31
neutral	0.55	0.87	0.67
positive	0.61	0.36	0.45

Table 6: Classification Report

We obtained an accuracy score of 56.79% after training the Random Forest model on the crowd sourced data.

3.2.2 Gold Data

Figure 4 shows the confusion matrix of the Random Forest model on the validation set after training the model on the gold data.

Table 11 shows the classification report of the Random Forest Classifier Model of the validation set after training on the gold data.

We obtained an accuracy score of 60.11% after training the Random Forest on the gold data.

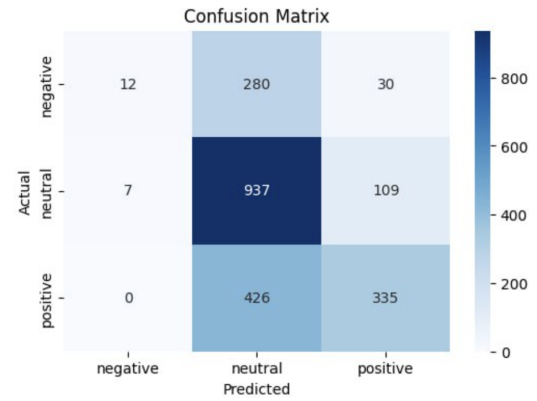


Figure 4: Random Forest Gold Data Confusion Matrix

Labels	Precision	Recall	F1 Score
negative	0.63	0.04	0.07
neutral	0.57	0.89	0.70
positive	0.71	0.44	0.54

Table 7: Classification Report

3.3 Multinomial Naive Bayes

We found that the best hyper parameters for the Multinomial Naive Bayes Model are 'alpha': 1, 'fit_prior': False for both training sets.

3.3.1 Crowd sourced Data

Figure 5 shows the confusion matrix of the Multinomial Naive Bayes model on the validation set after training the model on the crowd sourced data.

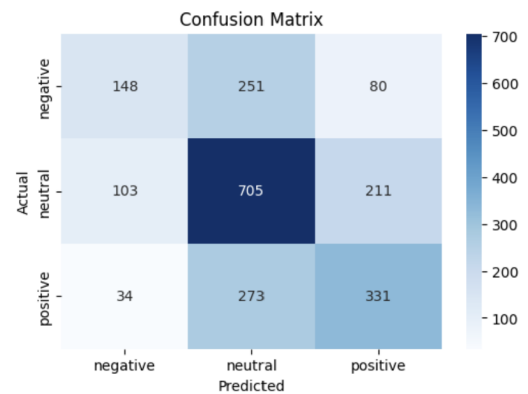


Figure 5: Multinomial Naive Bayes Crowd Sourced Data Confusion Matrix

Table 11 shows the classification report of the Multinomial Naive Bayes Model of the validation set after training on the crowd sourced data.

We obtained an accuracy score of 55.43% after training the Multinomial Naive Bayes model on

Labels	Precision	Recall	F1 Score
negative	0.52	0.31	0.39
neutral	0.57	0.69	0.63
positive	0.53	0.52	0.53

Table 8: Classification Report

the crowd sourced data.

3.3.2 Gold Data

Figure 6 shows the confusion matrix of the Multinomial Naive Bayes model on the validation set after training the model on the gold data.

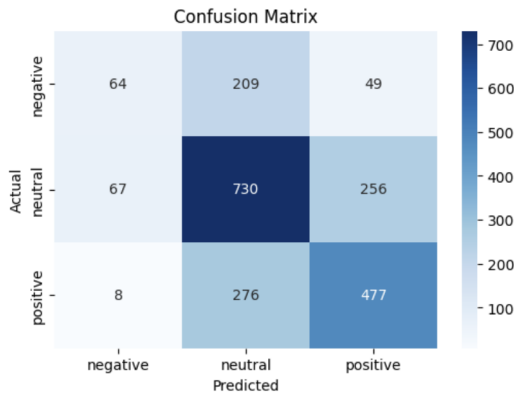


Figure 6: Multinomial Naive Bayes Gold Data Confusion Matrix

Table 11 shows the classification report of the Multinomial Naive Bayes Model of the validation set after training on the gold data.

Labels	Precision	Recall	F1 Score
negative	0.46	0.20	0.28
neutral	0.60	0.69	0.64
positive	0.61	0.63	0.62

Table 9: Classification Report

We obtained an accuracy score of 59.50% after training the Multinomial Naive Bayes model on the gold data.

3.4 Testing the best model

3.4.1 Results

In both the crowd sourced and gold dataset, the model that did the best was the Logistic Regression Model with the hyper parameters, 'C': 0.1, 'class_weight': None, 'penalty': 'l2', 'solver': 'liblinear'. We would use this model on the test set with the trusted labels.

Figure 7 shows the confusion matrix of the Logistic Regression Model on the trusted labels test set after training the model on the crowd sourced data.

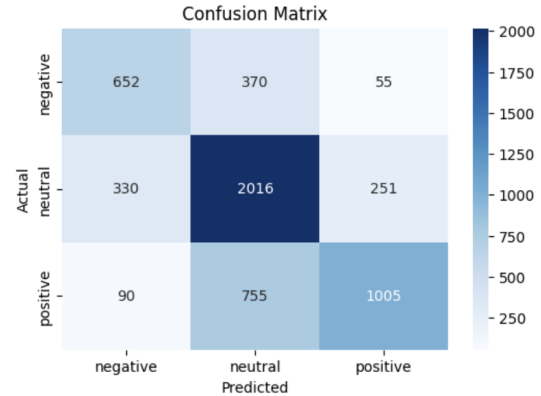


Figure 7: Logistic Regression Crowd Sourced Confusion Matrix

Table 11 shows the classification report of the Logistic Regression Model of the test set after training on the gold data.

Labels	Precision	Recall	F1 Score
negative	0.72	0.53	0.61
neutral	0.67	0.81	0.73
positive	0.77	0.66	0.71

Table 10: Classification Report

We obtained an accuracy score of 66.49% after training the Logistic Regression model on the crowd sourced data.

Figure 8 shows the confusion matrix of the Logistic Regression Model on the trusted labels test set after training the model on the gold data.

Table 11 shows the classification report of the Logistic Regression Model of the test set after training on the crowd sourced data.

Labels	Precision	Recall	F1 Score
negative	0.52	0.31	0.39
qwe			
neutral	0.57	0.69	0.63
positive	0.53	0.52	0.53

Table 11: Classification Report

We obtained an accuracy score of 70.22% after training the Logistic Regression model on the gold data.

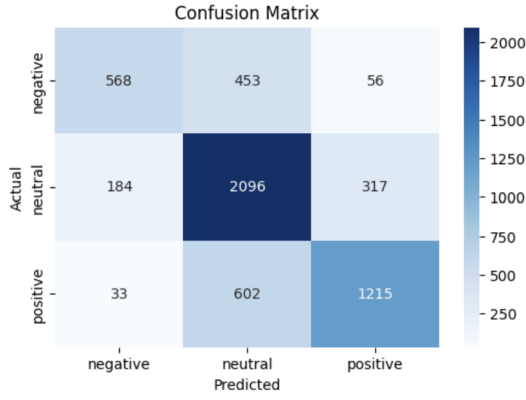


Figure 8: Logistic Regression Gold Confusion Matrix

3.4.2 Analysis of Results

With the current hyper parameters, we obtained an accuracy of 66.49% while training on crowd sourced data and an accuracy of 70.22% while training on gold data. A trivial baseline would have an accuracy of 33% and our model managed to obtain an accuracy twice as much as the trivial baseline. We know that our model is doing something 'smart' as it is able to accurately predict the sentiment of the text. We also know that the crowd sourced training data is reliable, as it achieved a Cohen's Kappa score of 0.4463 which implies that there is a moderate agreement between the 2 annotators. Some errors that the system makes would be classifying the text wrongly such as classifying it as 'Positive' when it is suppose to be 'Negative'. These errors can occur due to poor labeling of the text sentiments, having a poor selection of text used for training, texts that have conflicting or confusing sentences, the actual best hyper parameters for the model might not be the same hyper parameters we used, and the model functions based on numerical weights of features and these features can only represent one thing at a time, for example, bear can be both a noun and a verb, this can cause errors to occur.

4 Limitations

The limitations that we encountered are limited time and resources. With more time and resources, we could have experimented with various N-grams for TF-IDF vectorization, used more complex models such as k-nearest neighbors or gradient boosting classifier, used more values inside the hyper parameter grid for each model. We

could have possibly gotten better results if we had set the hyper parameters to other values, however, with the assumption that the hyper parameters values are continuous, we would need to test a range of values, which requires a lot of time and resources. Furthermore, while training on the crowd sourced data, we have a Cohen's Kappa score of 0.4463, this meant that there were disagreements in some of the text sentiment between the 2 annotators, this could lead to errors while training the models as well.

5 Conclusion

In our experiment, we successfully built a system capable of classifying text classification, and we trained our system on both the crowd sourced data and gold data. Our best model is the Logistic Regression model that trained on the gold data set and achieved an accuracy of 70.22% on the test set. Given the limitations we encountered on this experiment, we managed to create a system that achieved a decent result. However, for further continuation of this system, more complex models could be used, or more hyper parameters tuning could be done.